

1. Propósito del laboratorio

El laboratorio consiste en diseñar, comparar y desplegar un sistema de análisis binario de sentimientos sobre Sentiment140. El trabajo se centra en justificar, mediante evidencia experimental, las decisiones de preprocesamiento, representación del texto y clasificación que conforman el pipeline final.

El equipo partirá de dos referencias comunes, evaluará configuraciones bajo un mismo protocolo de validación cruzada, realizará un análisis de ablación sobre la solución seleccionada y, al final, reentrenará el modelo con todo el conjunto de entrenamiento para desplegarlo en AWS. MLflow se utilizará para registrar las comparaciones y mantener la trazabilidad entre ellas, la configuración elegida y el modelo publicado.

Criterio general de evaluación. Se evaluarán el proceso experimental, la trazabilidad de los resultados, el funcionamiento del modelo desplegado y el cumplimiento del contrato técnico definido en el Anexo A.

La guía fija los datos, el protocolo de comparación y la evidencia mínima que debe quedar registrada. El equipo conserva la decisión sobre cómo implementar las transformaciones que se compararán: qué negadores preservar, cómo tratar alargamientos y emojis, qué modelo spaCy usar, cómo obtener un vector por documento, qué hiperparámetros emplear y cuál será la configuración final.

El Anexo A reúne las convenciones y formatos exactos utilizados por la evaluación automática.

2. Protocolo experimental

Para que las comparaciones sean válidas, todas deberán utilizar el mismo conjunto de datos, la misma muestra y los mismos folds.

Dataset. Utilice exclusivamente el dataset de Hugging Face `adilbekovich/Sentiment140Twitter`, en la revisión indicada a continuación:

`b6037e127257d95b9b23d31f78b264b9ebe697fd`. Con la biblioteca `datasets`, cárguelo fijando esa revisión mediante `load_dataset(..., revision=...)`. En esta revisión, `train` contiene 1.360.000 registros y `test` 240.000; 0 corresponde a `negative` y 1 a `positive`. El conjunto `test` se reserva para la evaluación final.

Muestra y folds. A partir de `train`, obtenga una muestra estratificada de 200.000 registros con semilla 42. Conserve el índice original de `train` antes de muestrear o reindexar. Sobre esa muestra construya 3 folds estratificados, con mezcla aleatoria y semilla 42. La muestra y los folds se fijan antes de iniciar la experimentación y no podrán cambiar durante el laboratorio. Los equipos no tienen que obtener exactamente los mismos 200.000 registros. Para la evaluación, `protocol/partitions.csv` es la referencia que determina qué registros integran la muestra y en qué fold actúa cada uno como validación.

Métrica de comparación. Cada configuración se evalúa en los tres folds. Registre el Macro-F1 de cada fold, su promedio y la desviación estándar poblacional (`ddof=0`). Macro-F1 es el promedio del F1 de las clases `negative` y `positive`. Una configuración debe conservar los mismos hiperparámetros en los tres folds.

Registro del protocolo. En MLflow, un run es una unidad de registro que puede contener parámetros, métricas, tags y artefactos. Los runs que evalúan configuraciones se denominan aquí runs experimentales. Antes de ejecutar T0, B0 o cualquier otra configuración, cree un único run destinado a registrar las condiciones comunes del protocolo experimental: dataset, muestra, folds e integrantes. Este run no entrena ningún modelo; funciona como referencia para los runs experimentales posteriores. Los nombres exactos de los parámetros y artefactos se especifican en A.2.

Referencias iniciales.

- **T0:** en cada fold predice la clase más frecuente en la porción de entrenamiento; en caso de empate predice `negative`.
- **B0:** convierte a minúsculas, sustituye URL y menciones por los tokens `url` y `user`, normaliza espacios, utiliza BoW con unigramas y Logistic Regression. No aplica otras transformaciones. B0 utiliza los hiperparámetros por defecto de la biblioteca. Si durante su ejecución aparece una advertencia de convergencia, no modifique sus hiperparámetros para eliminarla en B0 ni durante las comparaciones de preprocesamiento de la Sección 3; cualquier ajuste de hiperparámetros deberá evaluarse por separado como una configuración adicional.

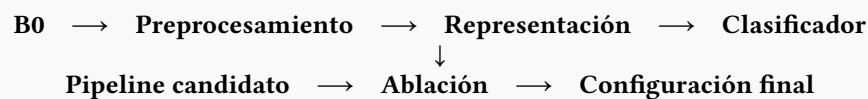
Evalúe T0 y B0 con el protocolo anterior antes de iniciar las comparaciones por etapas.



3. Comparaciones obligatorias

La experimentación se organiza en tres etapas. En cada una se modifica un componente del pipeline mientras los demás permanecen fijos. La configuración seleccionada se utiliza como base de la etapa siguiente. Los códigos P_*, R_* y C_* identifican las comparaciones y son valores del tag propio lab_experiment_id de cada run, no del experiment_id nativo de MLflow. Para configuraciones adicionales use lab_experiment_id=EXTRA.

Etapa	Comparaciones obligatorias	Condición fija
Preprocesamiento	Frente a B0 evalúe por separado: eliminar stopwords de spaCy (P_STOPWORDS); eliminar stopwords preservando los negadores definidos por el equipo (P_STOPWORDS_NEGATION); lematizar (P_LEMMA); normalizar alargamientos (P_ELONGATION); y convertir emojis/emoticonos a texto (P_EMOJI). Después seleccione el preprocesamiento para la etapa siguiente; puede conservar B0 o evaluar una combinación adicional.	BoW unigramas + Logistic Regression.
Representación	Con el preprocesamiento seleccionado compare: BoW unigramas (R_BOW); TF-IDF unigramas (R_TFIDF_UNI); TF-IDF unigramas+bigrams (R_TFIDF_UNI_BI); y embeddings de spaCy con vectores preentrenados (R_SPACY).	Preprocesamiento + Logistic Regression.
Clasificador	Con la representación seleccionada compare Logistic Regression (C_LOGREG), Linear SVM (C_LINEAR_SVM) y SGDClassifier (C_SGD).	Datos, folds, preprocesamiento y representación.



Registre cada comparación como un run experimental y evalúela con los tres folds. Las secciones A.2 a A.4 definen los requisitos técnicos, los identificadores y la configuración esperada para cada comparación.

Representación con spaCy. En R_SPACY, el modelo seleccionado deberá disponer de vectores preentrenados adecuados al método definido por el equipo.

Contribución individual. Cada integrante registrará al menos 3 configuraciones distintas que se contabilicen para su contribución, distribuidas en al menos 2 de estas etapas: preprocesamiento, representación, clasificador y ablación. T0 y B0 no cuentan para este mínimo. Algunas comparaciones obligatorias pueden corresponder a la misma configuración del pipeline, por lo que no todas aumentan este conteo; si es necesario, utilice configuraciones EXTRA válidas para alcanzar el mínimo individual.

4. Selección y análisis de ablación

Una vez completadas las comparaciones obligatorias, seleccione un **pipeline candidato** con base en los resultados de validación cruzada. El conjunto test no se utiliza para esta decisión.

Ablación. Revierta, una a la vez, las decisiones del candidato que difieren de B0 y mantenga fijo el resto del pipeline. Compare el Macro-F1 de cada ablación con el del candidato. Por ejemplo, si el candidato utiliza TF-IDF con unigramas+bigrams y Linear SVM, una ablación de la representación vuelve a la representación BoW de B0 sin modificar las demás decisiones.

A.4 define las seis decisiones que pueden someterse a ablación y la forma de registrar cada reversión. Si el candidato no difiere de B0 en ninguna de ellas, no se exige ablación; si difiere en una, evalúe una; si difiere en dos o más, evalúe al menos dos.

Después de las ablaciones, el equipo puede mantener el candidato o elegir una configuración revisada. En el segundo caso, evalúe esa configuración con los mismos tres folds y regístrela como EXTRA antes de definirla como final. No se exige una segunda ronda de ablación.

5. Modelo final y análisis de errores

Después de seleccionar la configuración final, reentrene el pipeline con los 1.360.000 registros de train y evalúelo una sola vez sobre los 240.000 registros de test. No ajuste la configuración a partir del resultado obtenido en test.

El run final deberá referenciar el run experimental que sustenta la configuración elegida y que cumple los requisitos del Anexo A. Registre el modelo en MLflow Model Registry con nombre exacto sentiment140 y alias champion. El modelo registrado deberá encapsular el preprocesamiento, la representación y el clasificador necesarios para recibir texto sin procesar y devolver la etiqueta de sentimiento.



Análisis de errores. Seleccione aleatoriamente al menos 20 errores del modelo final usando semilla 42 e incluya errores de ambas clases cuando existan. Clasifique cada caso, según el fenómeno predominante, en una de las categorías definidas en A.6; resuma la frecuencia por categoría e interprete los dos patrones más frecuentes. Si todos los errores pertenecen a una sola categoría, interprete esa categoría. Los formatos de los archivos se definen en A.6.

6. Registro en MLflow y despliegue

MLflow será el registro oficial de la experimentación y del modelo final. Todos los runs del equipo deberán agruparse en el MLflow Experiment `nlp-lab2-sentiment140`, que contendrá el protocolo, las configuraciones, las métricas, la procedencia y el run final. La evaluación tomará como evidencia lo registrado allí; reportes o copias locales no lo sustituyen.

El Tracking Server deberá ser compatible con Model Registry y alias. Use la misma versión de MLflow durante la experimentación y el registro final. En este laboratorio se utilizarán alias del Model Registry, no los stages heredados Staging o Production.

Tipo de run	Registro esperado
Protocolo	Datos, folds e integrantes comunes al equipo. No entrena un modelo.
Experimental	Una configuración evaluada mediante 3-fold CV, incluidas T0, B0 y las ablaciones.
Final	Reentrenamiento de la configuración seleccionada, evaluación sobre test y registro del modelo.

Un run incompleto no se considerará evidencia de una comparación realizada. Los tags, parámetros, métricas y artefactos obligatorios se encuentran en A.2.

Procedencia. Cada integrante ejecutará sus runs experimentales desde el SageMaker Notebook Instance asignado por el curso. No se aceptarán runs presentados desde SageMaker Studio o Google Colab. La procedencia debe quedar registrada en MLflow para atribuir las contribuciones individuales.

API y despliegue. La API FastAPI y el Tracking Server deberán estar accesibles durante la ventana de evaluación. Los endpoints de auditoría consultarán MLflow en tiempo de ejecución y el evaluador deberá poder recuperar desde MLflow los artefactos obligatorios. Para realizar inferencia, la API resolverá el modelo desde `sentiment140@champion`; un archivo serializado local o una copia incluida en el repositorio no se aceptarán como fuente del modelo.

Endpoint	Resultado esperado
POST /api/v1/predict	Ejecutar inferencia con el modelo registrado.
GET /audit/protocol	Consultar el protocolo experimental.
GET /audit/runs	Consultar los runs presentados.
GET /audit/contributions	Consultar la contribución de cada integrante.
GET /audit/model	Identificar el modelo desplegado y su trazabilidad.
GET /health	Verificar que el modelo pueda resolverse y atender inferencia.

Los contratos exactos de entrada, salida, tipos, errores, orden y tiempos se encuentran en A.5.

7. Uso de inteligencia artificial generativa

Se permite el uso de IA generativa y agentes especializados para programación, experimentación, pruebas, análisis, documentación, despliegue y resolución de errores. El equipo es responsable de comprobar el código, la configuración y los resultados incorporados a la solución.

- No comparta con herramientas externas contraseñas, claves, tokens, credenciales ni información sensible de AWS Academy.
- En el README indique las herramientas de IA utilizadas y su propósito.
- El clasificador final deberá ser entrenado por el equipo con los datos del laboratorio. No se aceptan APIs externas ni clasificadores previamente ajustados para resolver directamente esta tarea. Se permiten embeddings preentrenados como representación.



8. Evaluación automatizada y sustentación

La evaluación automática consultará la API, el Tracking Server y el Model Registry. Se verificarán las comparaciones obligatorias, la correspondencia entre etapas y configuraciones, la consistencia aritmética de las métricas registradas, las contribuciones individuales, la procedencia en SageMaker y la trazabilidad del modelo desplegado. En particular, se comprobarán la media y la desviación estándar poblacional de los tres folds y, cuando aplique, la métrica de ablación `macro_f1_delta`, definida en A.2:

`/api/v1/predict` → `sentiment140@champion` → `run final` → `run experimental` seleccionado.

Se utilizarán textos adicionales no publicados para calcular un Macro-F1 oculto mediante `/api/v1/predict`. Este componente equivale al 15 % de la calificación. Su fracción de puntaje será:

$$s = \text{clip} \left(\frac{F1_{\text{est}} - F1_{B0}}{F1_{\text{ref}} - F1_{B0}}, 0, 1 \right).$$

$F1_{\text{est}}$ es el Macro-F1 obtenido por el modelo del equipo sobre el conjunto oculto. $F1_{B0}$ es el resultado del baseline B0 del evaluador y $F1_{\text{ref}}$ el de una configuración clásica de referencia sometida a las mismas restricciones, con $F1_{\text{ref}} > F1_{B0}$. Ambos valores se publicarán antes de la ventana de evaluación; los textos y etiquetas ocultos no se publicarán. Igualar o superar $F1_{\text{ref}}$ otorga el puntaje completo de este componente.

El Macro-F1 oculto se calificará únicamente si el modelo puede trazarse hasta un `run final` válido y hasta `sentiment140@champion`. El `test_macro_f1` registrado por el equipo y el Macro-F1 oculto corresponden a conjuntos distintos y no tienen que coincidir.

Sustentación. `notebooks/experiment_audit.ipynb` se utilizará como apoyo y no forma parte del contrato de caja negra. Al ejecutarse contra el Tracking Server deberá permitir reconstruir los runs, el protocolo y las contribuciones. Durante la sustentación se podrá solicitar una modificación acotada. El uso de agentes de IA continuará permitido y el resultado podrá comprobarse mediante pruebas adicionales.

9. Entregables

- URL pública de la API, URL del MLflow Tracking Server accesible al evaluador y enlace al repositorio. Si MLflow requiere autenticación, entregue credenciales de solo lectura por el canal definido para la evaluación; no las incluya en el repositorio.
- Runs, artefactos y modelo `sentiment140@champion` registrados en MLflow conforme al Anexo A.
- `notebooks/experiment_audit.ipynb` para la sustentación.
- README, `reports/error_analysis.csv` y `reports/error_analysis.md`; los archivos de análisis deberán estar también registrados como artefactos del `run final`.

10. Verificación antes de la entrega

Antes de cerrar la entrega, compruebe lo siguiente:

- El `run` de protocolo existe, es único y corresponde a la muestra y los folds utilizados en todas las comparaciones.
- T0, B0 y todas las comparaciones obligatorias están registradas y utilizan los mismos tres folds.
- Cada integrante cumple el mínimo de configuraciones y etapas exigido para la contribución individual.
- La configuración final fue seleccionada sin utilizar `test`; el modelo se reentrenó con todo `train` y `test` se evaluó una sola vez.
- El alias `sentiment140@champion` resuelve el modelo asociado al `run final` válido y la API utiliza ese mismo modelo para inferencia.
- La API y el Tracking Server son accesibles desde una red externa y los endpoints de auditoría consultan MLflow en tiempo de ejecución.
- Los archivos de análisis de errores están completos y registrados como artefactos del `run final`.
- El repositorio no contiene credenciales, claves ni tokens, y el README declara las herramientas de IA utilizadas.



Anexo A. Contrato técnico para evaluación automática

El Anexo A reúne los nombres de campos, convenciones, formatos y respuestas que utiliza el evaluador automático. **No añade actividades al laboratorio.** Los valores marcados como ilustrativos deberán sustituirse por los valores reales del equipo.

A.1. Identificadores y vocabulario controlado

Nombre	Definición normativa
T0	Referencia trivial definida en la Sección 2.
B0	Baseline experimental definido en la Sección 2 y en A.3.
Run de protocolo	Único run que fija el contexto común antes de T0, B0 o cualquier run experimental. Su <code>run_id</code> se referencia desde los runs posteriores mediante <code>lab_protocol_run_id</code> .
<code>configuration_id</code>	Identificador definido por el equipo para una configuración efectiva (por ejemplo, CFG_001). El valor no codifica información obligatoria; runs con contenido equivalente en <code>run/configuration.json</code> deberán compartir exactamente el mismo identificador.
Pipeline candidato	Configuración seleccionada después de las comparaciones obligatorias y antes de la ablación.
Ablación	Evaluación del candidato en la que se revierte una decisión reconocida y se mantienen sin cambio las demás.
Run final	Run que registra el reentrenamiento con todo <code>train</code> , la evaluación sobre <code>test</code> y la versión del modelo con alias <code>champion</code> .

Convención de nombres. Los tags `lab_*` son propios del laboratorio: `lab_experiment_id` no es el `experiment_id` nativo de MLflow, `lab_stage` no es un stage del Model Registry y los tags `lab_*_run_id` guardan el `run_id` nativo que referencian.

Configuración efectiva. Contenido de `run/configuration.json` que describe el pipeline ejecutado; su equivalencia se define en A.3. **Run válido.** Run presentado, en estado `FINISHED`, con los campos obligatorios de su tipo, que cumple las reglas de configuración aplicables y, cuando corresponda, la procedencia definida en este anexo.

Todos los runs presentados deberán pertenecer al MLflow Experiment de nombre exacto `nlp-lab2-sentiment140`. Un run se considera presentado cuando tiene el tag reservado `lab_run_type` con valor `protocol`, `experiment` o `final`. Los runs exploratorios sin ese tag serán ignorados.

Los valores de `lab_run_type`, `lab_stage` y `lab_experiment_id` son sensibles a mayúsculas y minúsculas. Se permiten tags, params, metrics y artefactos adicionales; también pueden aparecer tags automáticos de MLflow.

A.2. Contrato de los runs de MLflow

Run de protocolo. Debe existir exactamente uno con `lab_run_type=protocol`. Registrará estos params exactos:

- `dataset_id=adilbekovich/Sentiment140Twitter;`
- `dataset_revision=b6037e127257d95b9b23d31f78b264b9ebe697fd;`
- `sampling_strategy=stratified, sample_size=200000, random_seed=42;`
- `cv_strategy=StratifiedKfold, cv_folds=3, cv_shuffle=true.`

Registrará los artefactos UTF-8 `protocol/partitions.csv` y `protocol/members.csv`. Son rutas lógicas relativas a la raíz de artefactos del run, no rutas del repositorio ni URI físicos.

`protocol/partitions.csv` tendrá encabezado exacto `index, fold`, 200.000 filas de datos, una por registro de la muestra, sin índices repetidos y ordenadas ascendentemente por `index`. `index` es la posición original en el split `train`, comenzando en 0, y deberá conservarse aunque se reordene o reindexe la muestra. `fold=k` significa que ese registro actúa como validación en el fold `k` y como entrenamiento en los otros dos. Este artefacto es la referencia utilizada por el evaluador para reconstruir la muestra y los folds; `random_seed=42` documenta la regla usada para generarlos.

`protocol/members.csv` tendrá encabezado exacto `member_id, notebook_arn`, una fila por integrante y orden lexicográfico ascendente por `member_id`. El `member_id` será el identificador suministrado por el curso y deberá usarse sin modificaciones.

Run experimental. T0 y B0 se registran también como runs experimentales.

- Tags: `lab_run_type=experiment`, `lab_protocol_run_id`, `lab_experiment_id`, `lab_stage`, `lab_member_id`, `lab_configuration_id`, `notebook_arn`.
- Métricas: `macro_f1_fold_0`, `macro_f1_fold_1`, `macro_f1_fold_2`, `macro_f1_mean`, `macro_f1_std`.



- Artefactos: `run/configuration.json` y `provenance/sagemaker-resource-metadata.json`.
- Ablación: además, tag `lab_ablation_parent_run_id`, param `ablation_reverted_decision` y métrica `macro_f1_delta`.

Run final.

- Tags: `lab_run_type=final`, `lab_protocol_run_id`, `lab_selected_experiment_run_id`, `lab_configuration_id`, `lab_member_id`, `notebook_arn`.
- Param: `training_size=1360000`.
- Métrica: `test_macro_f1`.
- Artefactos: `run/configuration.json`, `provenance/sagemaker-resource-metadata.json`, `reports/error_analysis.csv` y `reports/error_analysis.md`.

El run final deberá apuntar a un run experimental válido mediante `lab_selected_experiment_run_id`. Ambos deberán compartir `lab_configuration_id` y configuraciones equivalentes.

Métricas derivadas. `macro_f1_mean` y `macro_f1_std` corresponderán a la media y desviación estándar poblacional de los tres folds. En ablación:

$$\text{macro_f1_delta} = \text{macro_f1_mean}(\text{parent}) - \text{macro_f1_mean}(\text{ablation}).$$

Procedencia. En un SageMaker Notebook Instance, AWS expone este archivo en `/opt/ml/metadata/resource-metadata.json`; registre una copia sin editarla como `provenance/sagemaker-resource-metadata.json`. Su campo `ResourceArn` identificará el Notebook Instance asignado al integrante y coincidirá exactamente con el tag `notebook_arn`. El evaluador contrastará también `lab_member_id` y `notebook_arn` con la asignación oficial. Un run sin esta correspondencia no será válido.

A.3. Esquema de `run/configuration.json`

Todo run experimental y final registrará un JSON válido con exactamente tres campos de primer nivel: `preprocessing`, `representation` y `classifier`, y dentro de ellos solo los campos definidos a continuación. No incluirá nombres de integrantes, timestamps, run IDs ni información ajena a la configuración efectiva.

- `preprocessing`: objeto con `lowercase` (boolean), `url`, `mention`, `whitespace`, `stopwords`, `negators` (lista), `lemmatize` (boolean), `elongation`, `elongation_spec`, `emoji`, `emoji_spec`, `resources` (objeto) y `additional` (objeto). `resources` identifica modelos, bibliotecas o recursos externos que afecten el preprocesamiento y sus versiones. `additional` describe cualquier transformación adicional no representada por los otros campos.
- `representation`: objeto con `type`, `ngram_range`, `library`, `library_version`, `spacy_model`, `spacy_model_version` y `parameters`. `parameters` contiene los parámetros configurados explícitamente que afecten el representador. Si `type=spacy_embedding`, deberá incluir `document_vector_method` como string no vacío.
- `classifier`: objeto con `type`, `library`, `library_version` y `parameters`. `parameters` contiene los hiperparámetros configurados explícitamente. Los valores predeterminados no modificados quedan determinados por `library_version` y no necesitan expandirse.

Valores controlados.

- `url`, `mention`: `keep`, `drop` o `token:<valor>`;
- `whitespace`: `keep` o `normalize`;
- `stopwords`: `keep`, `remove` o `remove_preserve_negation`;
- `elongation`: `keep` o `normalize`;
- `emoji`: `keep` o `text`;
- `representation.type`: `bow`, `tfidf` o `spacy_embedding`;
- `classifier.type`: `logistic_regression`, `linear_svm`, `sgd` o `most_frequent`.

Campos no aplicables. Ningún campo obligatorio podrá omitirse. Cuando no aplique, `elongation_spec`, `emoji_spec`, `spacy_model` y `spacy_model_version` serán null; `resources`, `additional` y `parameters` serán objetos vacíos {}. `negators=[]` salvo con `stopwords=remove_preserve_negation`, cuando será una lista no vacía. `elongation_spec` será no nulo únicamente con `elongation=normalize`; `emoji_spec`, únicamente con `emoji=text`. Para `bow` y `tfidf`, `ngram_range` será una lista de dos enteros; para `spacy_embedding`, será [] y `parameters.document_vector_method` será un string no vacío. Como referencia, son válidas descripciones como `reduce_repeated_chars_to_2`, `emoji.demojize` o `mean_token_vectors`; son ejemplos ilustrativos, no valores obligatorios. Para T0, `preprocessing=null`, `representation=null` y `classifier.type=most_frequent`.

Equivalencia. Dos runs con configuraciones JSON equivalentes deberán compartir `lab_configuration_id`; cambiar solo ese identificador no crea una configuración distinta. Un mismo identificador podrá aparecer con distinto `lab_experiment_id` cuando



la configuración efectiva sea idéntica. El orden de claves de objetos JSON no afecta la equivalencia; el orden de listas sí.

Entorno de software. Mantenga estables las versiones de las bibliotecas durante la experimentación y el entrenamiento final. Si cambia una versión registrada en `library_version`, vuelva a ejecutar B0 y las configuraciones que dependan de esa biblioteca antes de compararlas o seleccionarlás.

En comparaciones definidas como *igual a B0, salvo...*, la igualdad se evaluará sobre todos los campos excepto `preprocessing.resources`, que reflejará los recursos realmente utilizados y podrá variar sin convertirse en variable experimental. Los demás campos, incluido `preprocessing.additional`, permanecerán iguales salvo indicación expresa.

Forma canónica de B0. `library` y `library_version` reflejarán la biblioteca y versión realmente usadas.

```
{
  "preprocessing": {
    "lowercase": true,
    "url": "token:url",
    "mention": "token:user",
    "whitespace": "normalize",
    "stopwords": "keep",
    "negators": [],
    "lemmatize": false,
    "elongation": "keep",
    "elongation_spec": null,
    "emoji": "keep",
    "emoji_spec": null,
    "resources": {},
    "additional": {}
  },
  "representation": {
    "type": "bow",
    "ngram_range": [1, 1],
    "library": "LIBRARY_USED",
    "library_version": "VERSION_USED",
    "spacy_model": null,
    "spacy_model_version": null,
    "parameters": {}
  },
  "classifier": {
    "type": "logistic_regression",
    "library": "LIBRARY_USED",
    "library_version": "VERSION_USED",
    "parameters": {}
  }
}
```

Forma canónica de T0.

```
{
  "preprocessing": null,
  "representation": null,
  "classifier": {
    "type": "most_frequent",
    "library": "LIBRARY_USED",
    "library_version": "VERSION_USED",
    "parameters": {}
  }
}
```

Ejemplo estructural para `spacy_embedding`. Los marcadores en mayúsculas son ilustrativos y no prescriben qué modelo, recursos o método debe seleccionar el equipo.

```
{
  "preprocessing": {
    "lowercase": true, "url": "token:url", "mention": "token:user",
    "whitespace": "normalize", "stopwords": "keep", "negators": [],
    "lemmatize": false,
    "elongation": "normalize",
    "elongation_spec": "SPEC_DEFINED_BY_TEAM",
    "emoji": "text",
    "emoji_spec": "SPEC_DEFINED_BY_TEAM",
    "resources": {},
    "additional": {}
  },
  "representation": {
    "type": "spacy_embedding",
    "ngram_range": [],
    "library": "spacy",
    "library_version": "VERSION_USED",
    "spacy_model": "MODEL_USED",
    "spacy_model_version": "MODEL_VERSION_USED",
    "parameters": {"document_vector_method": "METHOD_DEFINED_BY_TEAM"}
  },
  "classifier": {"type": "logistic_regression",
```

```

"library": "LIBRARY_USED", "library_version": "VERSION_USED",
"parameters": {}
}

```

A.4. Correspondencia de comparaciones y ablaciones

Los componentes no indicados como variables permanecerán iguales a la referencia de la etapa. Todos los runs R_* usarán el mismo preprocesamiento seleccionado y Logistic Regression. Todos los runs C_* usarán el mismo preprocesamiento y la misma representación seleccionados. El preprocesamiento seleccionado deberá coincidir con B0 o con un run válido de etapa preprocessing; la representación seleccionada deberá coincidir con un run válido de etapa representation.

lab_experiment_id	lab_stage	Restricción de configuración
T0	reference	Configuración T0 de A.3.
B0	baseline	Configuración B0 de A.3.
P_STOPWORDS	preprocessing	Igual a B0, salvo stopwords=remove.
P_STOPWORDS_NEGATION	preprocessing	Igual a B0, salvo stopwords=remove_preserve_negation y negators no vacío.
P_LEMMA	preprocessing	Igual a B0, salvo lemmatize=true.
P_ELONGATION	preprocessing	Igual a B0, salvo elongation=normalize y elongation_spec no nulo.
P_EMOJI	preprocessing	Igual a B0, salvo emoji=text y emoji_spec no nulo.
R_BOW	representation	Preprocesamiento seleccionado + type=bow, ngram_range=[1, 1] y Logistic Regression.
R_TFIDF_UNI	representation	Preprocesamiento seleccionado + type=tfidf, ngram_range=[1, 1] y Logistic Regression.
R_TFIDF_UNI_BI	representation	Preprocesamiento seleccionado + type=tfidf, ngram_range=[1, 2] y Logistic Regression.
R_SPACY	representation	Preprocesamiento seleccionado + type=spacy_embedding y Logistic Regression.
C_LOGREG	classifier	Preprocesamiento y representación seleccionados + type=logistic_regression.
C_LINEAR_SVM	classifier	Preprocesamiento y representación seleccionados + type=linear_svm.
C_SGD	classifier	Preprocesamiento y representación seleccionados + type=sgd.
ABLATION	ablation	Cantidad y reversión definidas a continuación; lab_ablation_parent_run_id identifica el run válido del candidato.
EXTRA	preprocessing, representation, classifier o ablation	Comparación adicional válida; no sustituye ninguna obligatoria.

Una decisión ablacionable se identifica mediante uno de estos seis valores exactos:

```

preprocessing.stopwords, preprocessing.lemmatize,
preprocessing.elongation, preprocessing.emoji,
representation, classifier.

```

Revertir una decisión significa devolver a B0 todos los campos que la forman y mantener sin cambio los demás. Para stopwords: stopwords=keep y negators=[]; para lematización: lemmatize=false; para alargamientos: elongation=keep y elongation_spec=null; y para emoji: emoji=keep y emoji_spec=null.

Revertir representation significa usar la forma de B0: type=bow, ngram_range=[1, 1], campos spaCy en null, parameters={} y los campos library y library_version iguales a B0. Revertir classifier significa type=logistic_regression, parameters={} y los mismos valores de library y library_version que B0. Cambios de hiperparámetros dentro de un mismo representador o clasificador no generan por sí solos una ablación obligatoria. El param ablation_reverted_decision usará uno de los seis valores anteriores.

A.5. Contrato de la API

La API se implementará con FastAPI. Los endpoints JSON responderán con application/json. Los campos definidos son obligatorios y pueden añadirse campos adicionales. En /audit/runs, params y tags reproducen strings de MLflow, metrics contiene números y pueden aparecer tags de sistema adicionales. En endpoints especializados, los tipos JSON coincidirán con los definidos aquí.

/api/v1/predict, /audit/model, /health y el alias champion deberán identificar el mismo run final.

Las listas de identificadores y rutas se ordenarán lexicográficamente, salvo predictions, que conserva el orden de entrada. Para cálculos derivados se usará tolerancia absoluta 10^{-6} . Si MLflow no está disponible, los endpoints /audit/* responderán HTTP 503 con detail=mlflow_unavailable; no reconstruirán respuestas desde una copia local.



Método y ruta	Contrato
POST /api/v1/predict	Entrada JSON con text: un string o una lista de 1–32 strings; cada texto, máximo 1.000 caracteres. Salida HTTP 200: model_run_id (string) y predictions (lista de strings negative o positive). Para un string, la lista tiene un elemento; para un lote, conserva cantidad y orden. El modelo se resuelve desde sentiment140@champion.
GET /audit/protocol	HTTP 200 solo si existe exactamente un protocolo presentado. Devuelve protocol_run_id (string), dataset_id (string), dataset_revision (string), sampling_strategy (string), sample_size (entero), random_seed (entero), cv_strategy (string), cv_folds (entero), cv_shuffle (boolean), partitions_artifact (string) y members_artifact (string). Las rutas serán exactamente protocol/partitions.csv y protocol/members.csv. Si no existe exactamente un protocolo presentado: HTTP 409, detail=protocol_not_unique.
GET /audit/runs	Devuelve runs, lista ordenada por run_id. Cada objeto contiene run_id y status (strings), run_type (string, igual al tag lab_run_type), params (objeto de strings), metrics (objeto de números), tags (objeto de strings), artifacts (lista de strings) y configuration (objeto o null). Incluye todos los runs presentados, incluso incompletos. artifacts enumera recursivamente las rutas de archivos, relativas a la raíz y en orden lexicográfico. Para el run de protocolo, o si run/configuration.json falta o no es JSON válido en un run experimental/final, configuration=null.
GET /audit/contributions	Devuelve members (ordenada por member_id), invalid_run_ids y unattributed_run_ids. Cada miembro contiene member_id, notebook_arn, run_ids, counted_run_ids, configuration_ids, stages y valid_configurations. run_ids incluye runs experimentales válidos; counted_run_ids excluye T0/B0 e incluye los válidos que cuentan para el mínimo; EXTRA puede incluirse. configuration_ids es la lista ordenada, sin duplicados, de los IDs en los runs contados; valid_configurations es su cardinalidad. stages contiene, sin duplicados y ordenados, los valores de lab_stage. El mínimo exige valid_configurations>=3 y al menos dos etapas. Los runs experimentales/finales inválidos aparecen en invalid_run_ids; unattributed_run_ids contiene los que no pueden asociarse válidamente a un par member_id/notebook_arn, y todo run no atribuible aparece también en invalid_run_ids.
GET /audit/model	Devuelve model_name (string), alias (string), version (entero), run_id (string), protocol_run_id (string), selected_experiment_run_id (string), configuration_id (string), configuration (objeto), training_size (entero) y test_macro_f1 (número). model_name=sentiment140, alias=champion. Si no existe el alias: HTTP 404, detail=champion_not_found; si no se asocia a un run final válido: HTTP 409, detail=champion_invalid.
GET /health	Resuelve sentiment140@champion y verifica inferencia. HTTP 200 si está disponible; HTTP 503 si no puede cargarlo o resolverlo. Los cuerpos exactos aparecen en A.6.

Para /api/v1/predict son inválidos: ausencia de text, null, string vacío o solo espacios, lista vacía, más de 32 elementos, textos de más de 1.000 caracteres, tipos distintos de string o una lista con algún elemento inválido. La solicitud completa producirá HTTP 4xx y no entregará resultados parciales. Una solicitud válida se completará en un máximo de 10 segundos.

A.6. Respuestas y formatos de referencia

Artefactos del protocolo.

```
member_id,notebook_arn
E01,arn:aws:sagemaker:REGION:ACCOUNT:notebook-instance/NOTEBOOK_E01
E02,arn:aws:sagemaker:REGION:ACCOUNT:notebook-instance/NOTEBOOK_E02

index,fold
3,1
17,0
28,2
41,0
```

El segundo bloque es una muestra; el archivo real contiene 200.000 filas de datos.

Respuesta de GET /audit/protocol.

```
{
  "protocol_run_id": "PROTOCOL_RUN_ID",
  "dataset_id": "adilbekovich/Sentiment140Twitter",
  "dataset_revision": "b6037e127257d95b9b23d31f78b264b9ebe697fd",
  "sampling_strategy": "stratified",
  "sample_size": 200000,
  "random_seed": 42,
  "cv_strategy": "StratifiedKFold",
  "cv_folds": 3,
  "cv_shuffle": true,
  "partitions_artifact": "protocol/partitions.csv",
  "members_artifact": "protocol/members.csv"
}
```

Respuesta de GET /audit/contributions.

```
{
  "members": [
    {
      "member_id": "E01",
      "notebook_arn": "arn:aws:sagemaker:REGION:ACCOUNT:notebook-instance/NOTEBOOK_E01",
      "run_ids": ["RUN_1", "RUN_2", "RUN_3", "RUN_B0"],
      "counted_run_ids": ["RUN_1", "RUN_2", "RUN_3"],
      "configuration_ids": ["CFG_1", "CFG_2", "CFG_3"],
      "valid_configurations": 3,
      "stages": ["classifier", "preprocessing", "representation"]
    }
  ],
}
```



```

"invalid_run_ids": ["RUN_INVALID"],
"unattributed_run_ids": []
}

```

Respuesta de GET /audit/model. La configuración abreviada del ejemplo usa B0 solo para mostrar la estructura; no implica que B0 deba seleccionarse.

```

{
  "model_name": "sentiment140",
  "alias": "champion",
  "version": 3,
  "run_id": "FINAL_RUN_ID",
  "protocol_run_id": "PROTOCOL_RUN_ID",
  "selected_experiment_run_id": "SELECTED_EXPERIMENT_RUN_ID",
  "configuration_id": "CONFIGURATION_ID",
  "configuration": {
    "preprocessing": {
      "lowercase": true, "url": "token:url", "mention": "token:user",
      "whitespace": "normalize", "stopwords": "keep", "negators": [],
      "lemmatize": false, "elongation": "keep", "elongation_spec": null,
      "emoji": "keep", "emoji_spec": null, "resources": {}, "additional": {}
    },
    "representation": {
      "type": "bow", "ngram_range": [1, 1], "library": "LIBRARY_USED",
      "library_version": "VERSION_USED", "spacy_model": null,
      "spacy_model_version": null, "parameters": {}
    },
    "classifier": {
      "type": "logistic_regression", "library": "LIBRARY_USED",
      "library_version": "VERSION_USED", "parameters": {}
    }
  },
  "training_size": 1360000,
  "test_macro_f1": 0.8123
}

```

Inferencia y salud.

```

{"text": "this movie was surprisingly good"}

```

```

{
  "model_run_id": "FINAL_RUN_ID",
  "predictions": ["positive"]
}

```

```

{"text": ["i loved it", "worst day ever", "not bad at all"]}

```

```

{
  "model_run_id": "FINAL_RUN_ID",
  "predictions": ["positive", "negative", "positive"]
}

```

GET /health, servicio disponible:

```

{"status": "ok", "model_run_id": "FINAL_RUN_ID"}

```

GET /health, modelo no disponible:

```

{"status": "unavailable", "model_run_id": null}

```

Ejemplos de solicitudes inválidas:

```

{}
{"text": null}
{"text": ""}
{"text": "  "}
{"text": []}
{"text": ["ok", ""]}
{"text": ["ok", 7]}

```

Forma de reports/error_analysis.csv. El archivo será CSV UTF-8 con el encabezado exacto mostrado a continuación. index es la posición original en test. Los campos con comas, comillas o saltos de línea se codificarán como CSV válido. Las etiquetas serán exclusivamente negative o positive; category será una de: negation, intensification, contrast, mixed, emoji, elongation, informal, hashtag, sarcasm, other.

```

index,text,true_label,predicted_label,category
1042,"not bad, actually pretty good",positive,negative,negation
2871,"yeah right, best service ever",negative,positive,sarcasm
3390,"soooo tired of this",negative,positive,elongation

```

Forma de reports/error_analysis.md. Resuma la frecuencia por categoría e interprete los dos patrones más frecuentes; si solo aparece una categoría, interprétela. No se exige plantilla adicional.